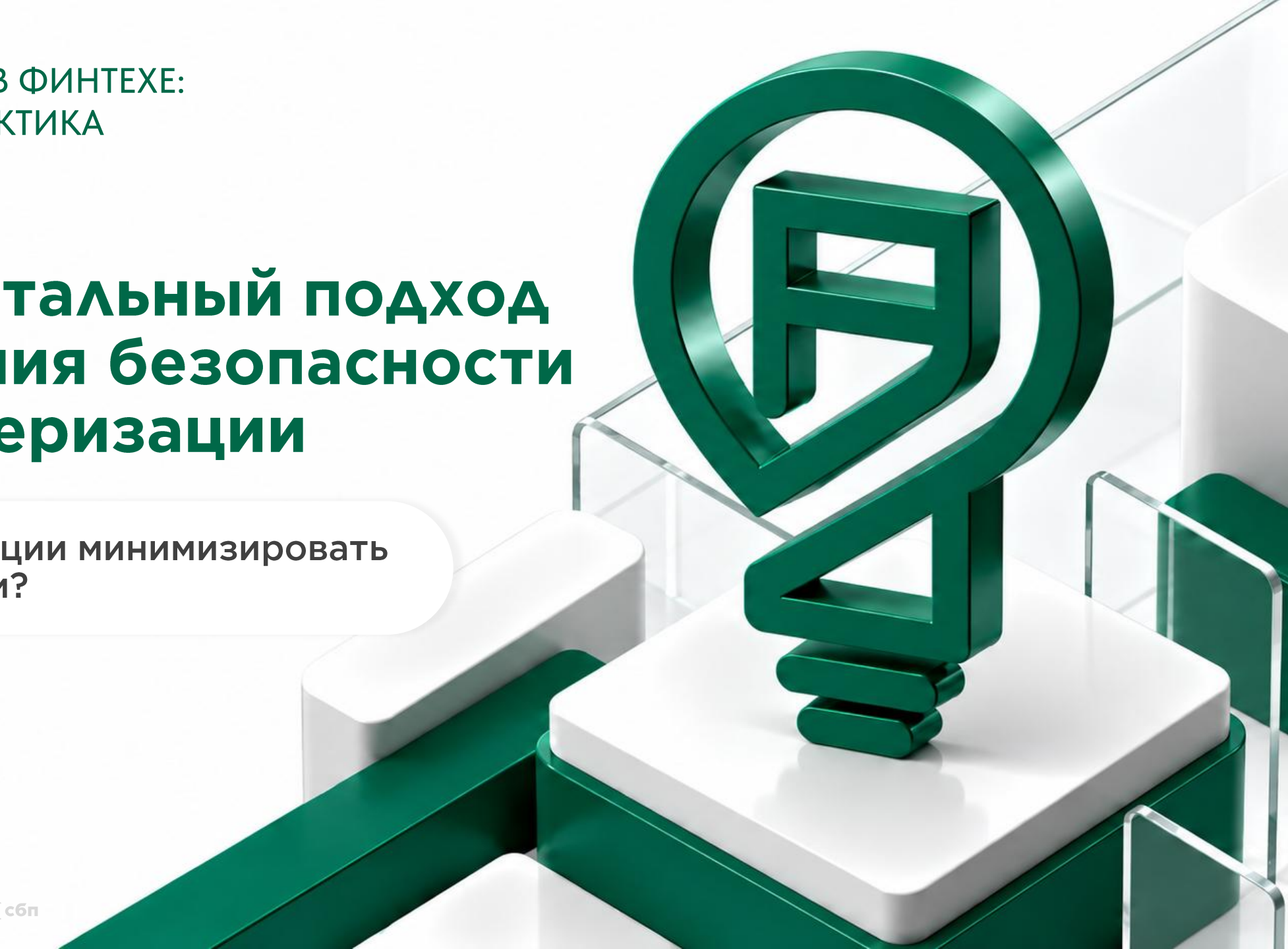
ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

Фундаментальный подход обеспечения безопасности в контейнеризации

Как при эксплуатации минимизировать поверхности атаки?

Юрий
Паркин

НСПК | МИР | СБП



План презентации

01 Контейнеризация

02 Угрозы и Defense in Depth

03 Namespaces и cgroups

04 Seccomp и AppArmor

05 Инструменты безопасности. Итоги



раздел

01

Контейнеризация



НСПК | МИР | СБП

История

1970

Unix v7, chroot
позволяет запустить процесс в изолированной среде, где он имеет доступ только к указанному корневому каталогу

2000

FreeBSD, Jails
изолирует не только файловую систему, но и сетевые интерфейсы и процессы

2002

Mount Namespace,
который изолировал список точек монтирования файловой системы

2005

Open VZ
реализовал контейнеризацию на уровне операционной системы

2007

Linux, Control groups (Google)
позволяют ограничивать и контролировать использование ресурсов (CPU, память, диск) для групп процессов

2008

LXC (Linux Containers)
обеспечивают изоляцию процессов, сетевых интерфейсов и файловых систем

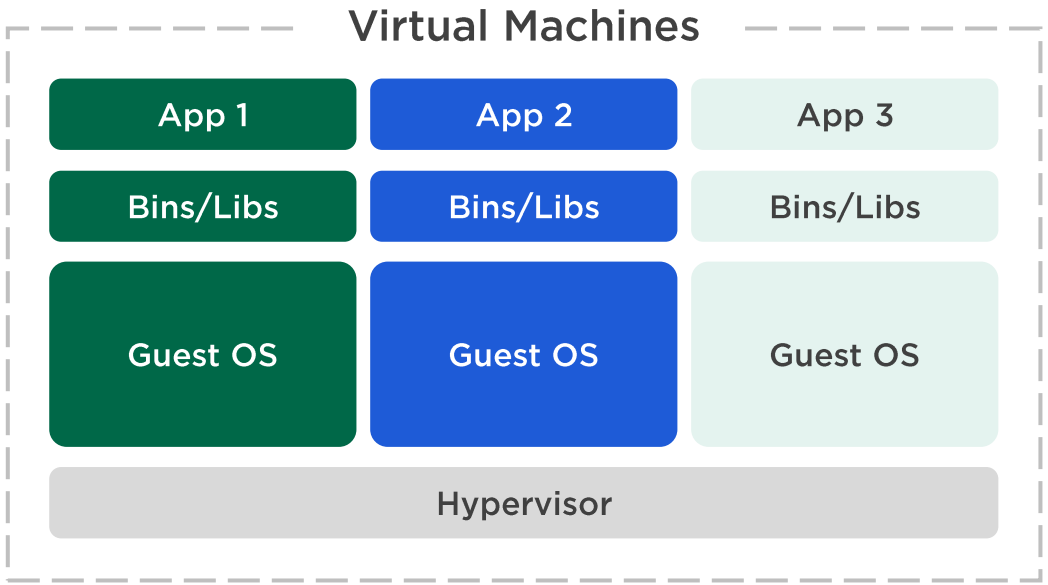
2013

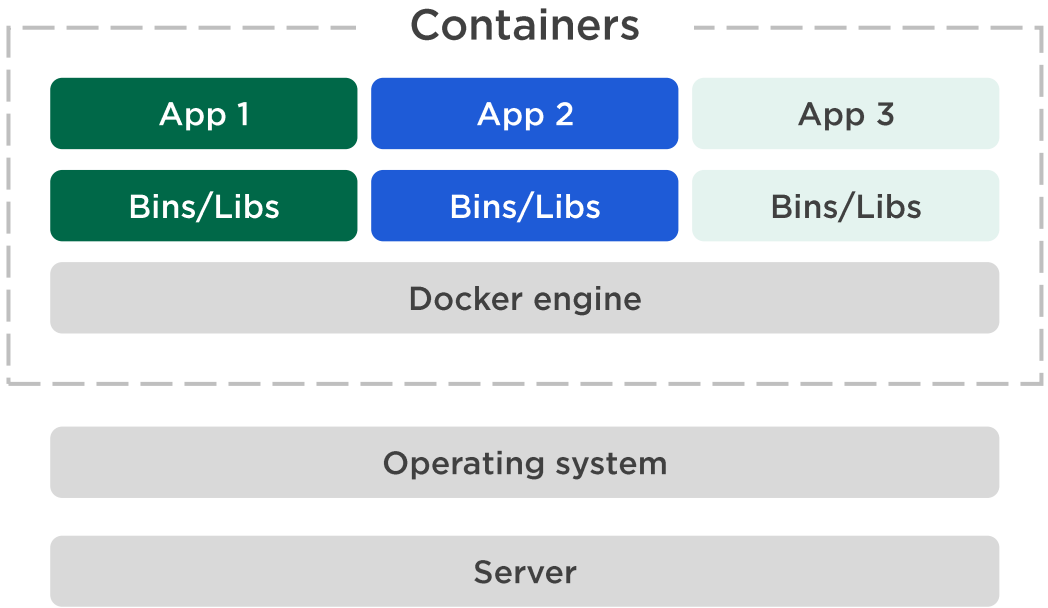
Docker release
упростил процесс создания, развертывания и управления контейнерами после появления Cgroups и Namespaces

2015

OCI RunC
стандартизация контейнеров, низкоуровневый runtime

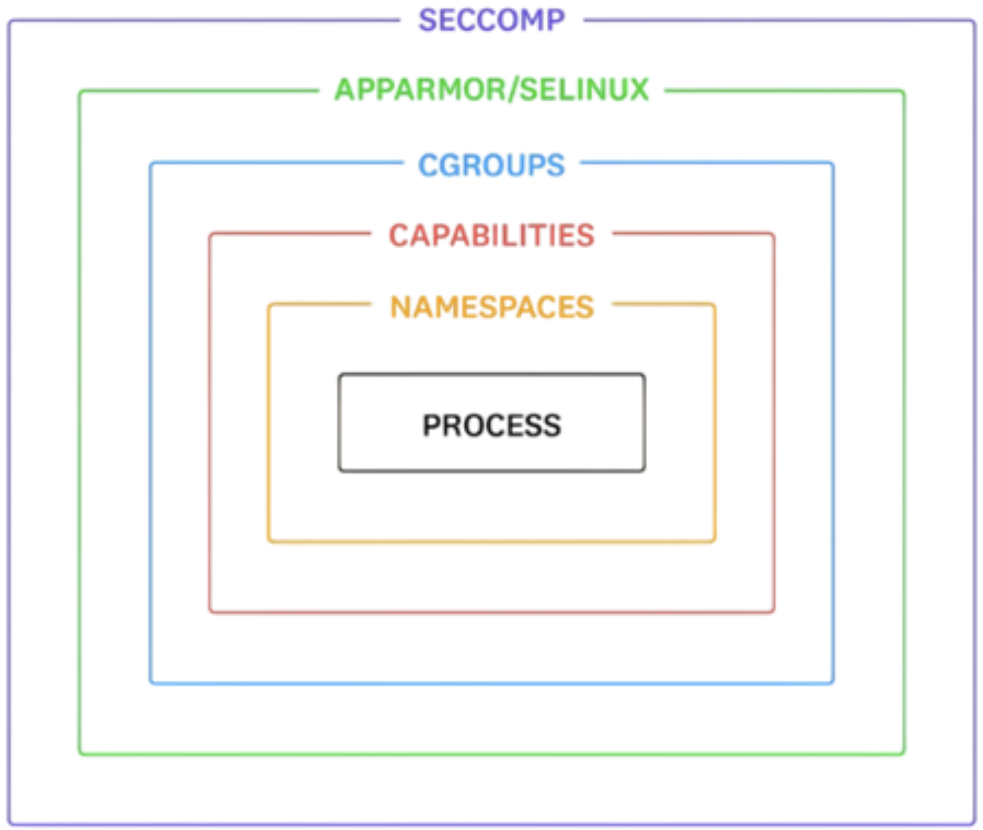
Контейнеризация и виртуализация



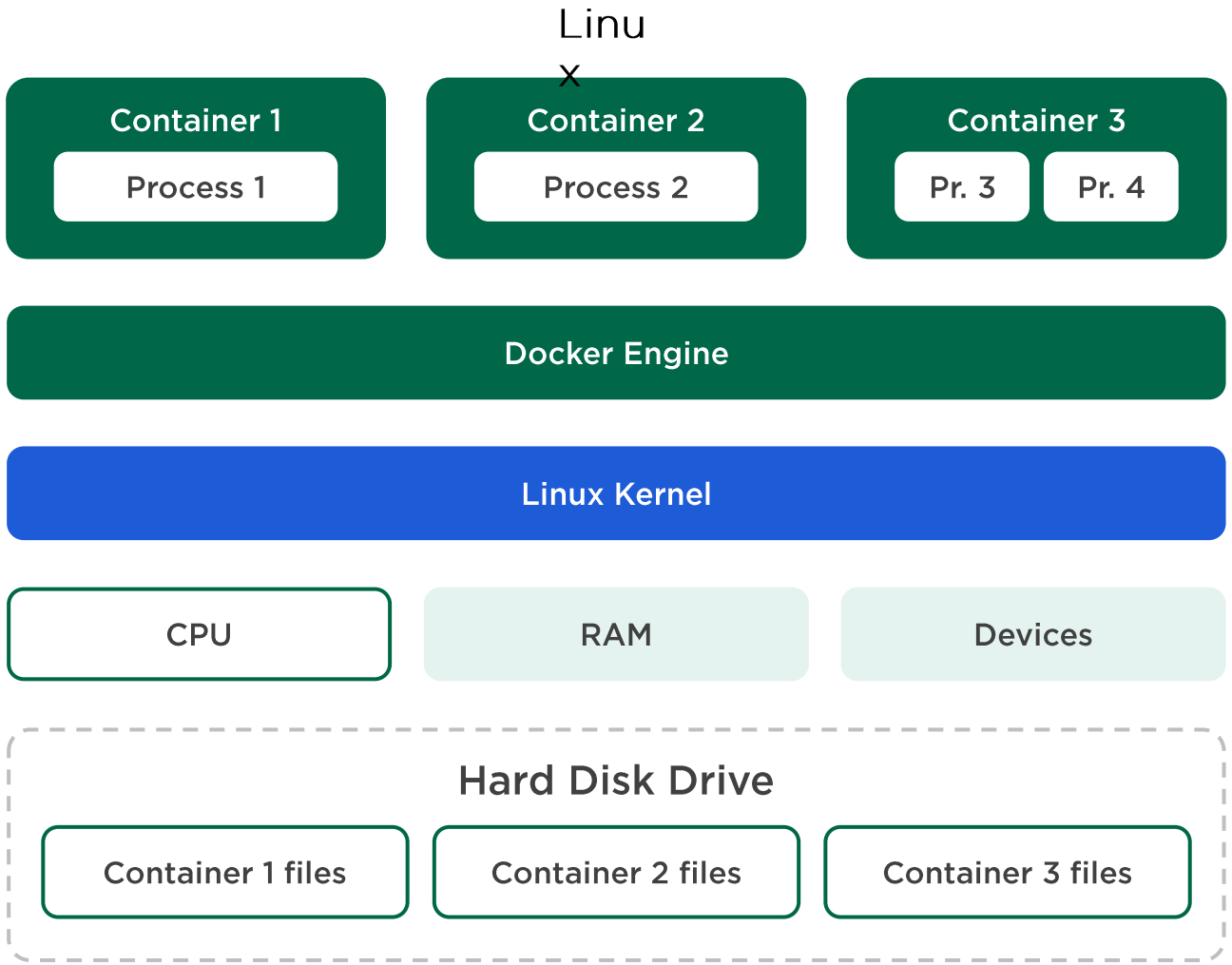


Что такое контейнер?

Контейнер — это изолированная среда для процесса, использующая механизмы безопасности ядра Linux



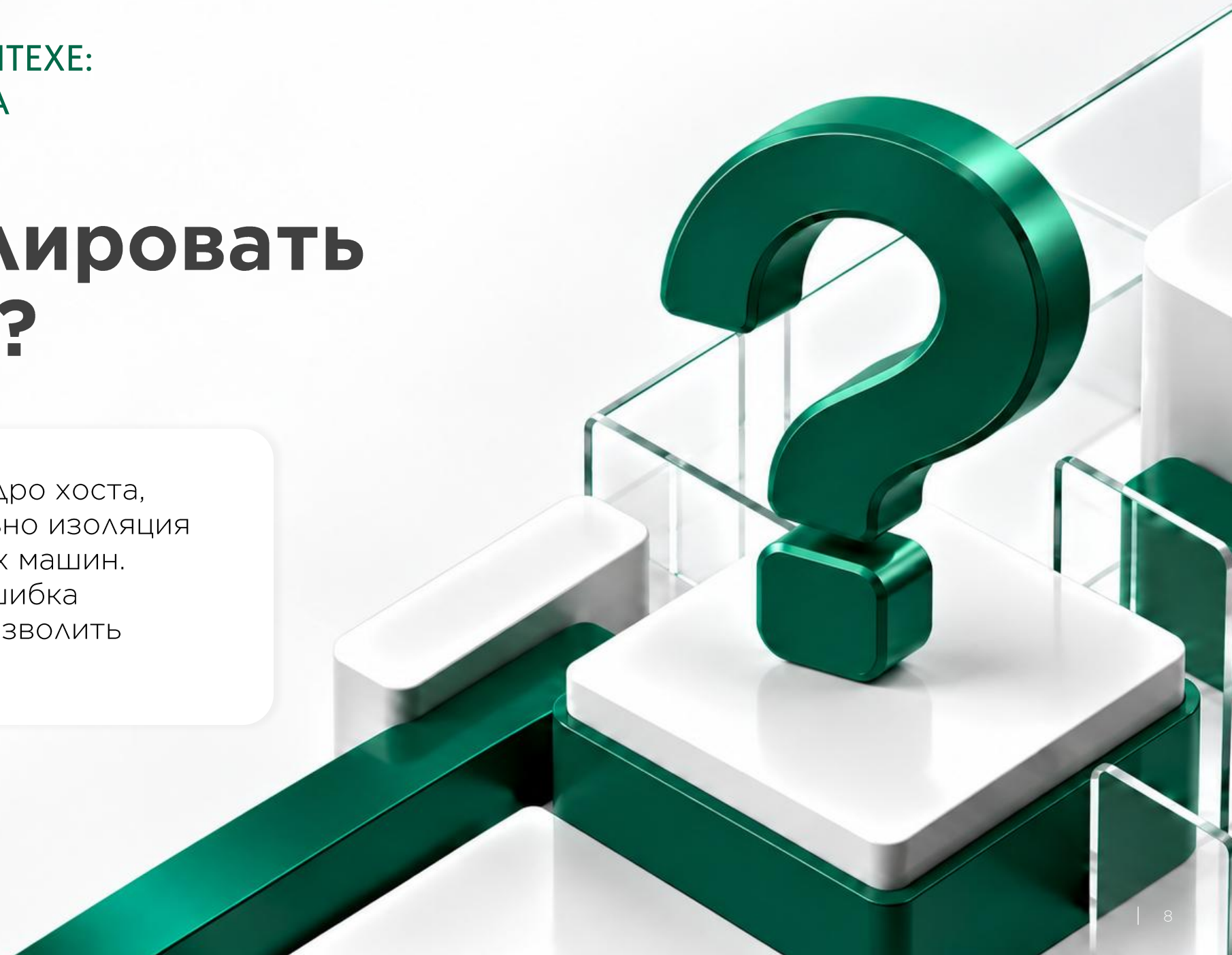
Как это работает?





Зачем изолировать контейнер?

Контейнеры разделяют ядро хоста, поэтому для них изначально изоляция слабее, чем у виртуальных машин. Уязвимость в ядре или ошибка в конфигурации может позволить «побег» из контейнера



раздел

02

Угрозы и Defense in depth



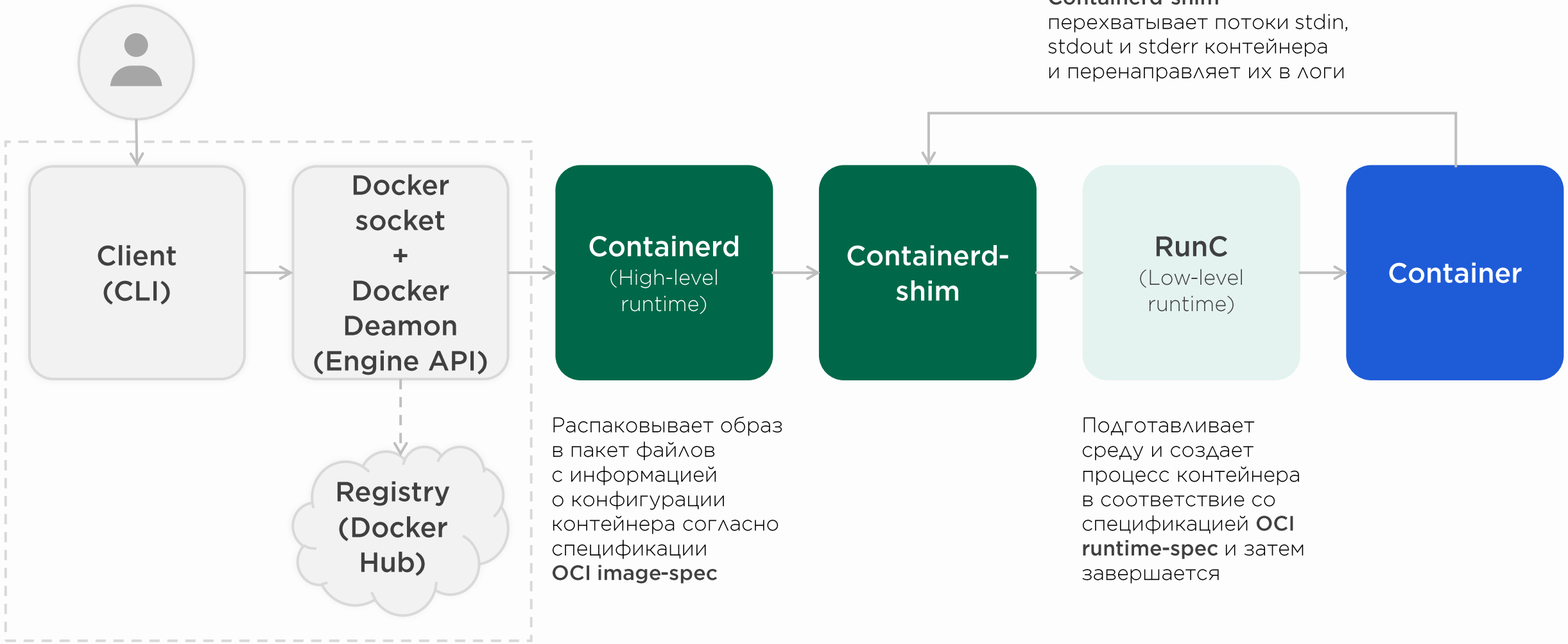
НСПК | МИР | СБП

Архитектура контейнеризации

NIST SP 800-190



Docker ver. > 1.11

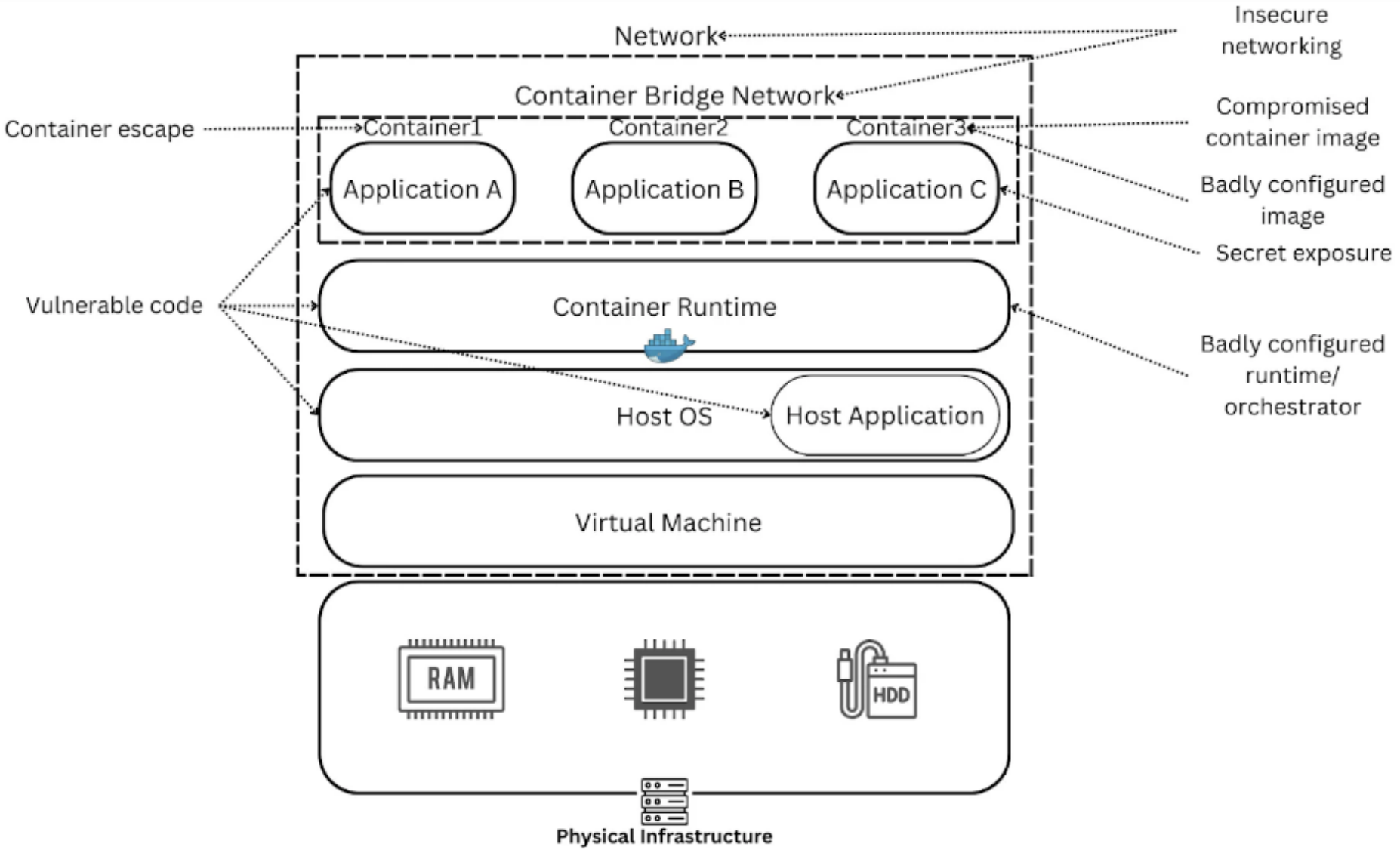


Containerd-shim
перехватывает потоки stdin, stdout и stderr контейнера и перенаправляет их в логи

Распаковывает образ в пакет файлов с информацией о конфигурации контейнера согласно спецификации **OCI image-spec**

Подготавливает среду и создает процесс контейнера в соответствие со спецификацией **OCI runtime-spec** и затем завершается

Основные угрозы



Container Security Layers



Запуск процесса as NON-root user

Описание

Docker Engine по умолчанию стартует процессы контейнера от root. Один и тот же процесс с UID 0 имеет права root как в контейнере, так и на хосте

Минимизация:

- защита от эскалации привилегий слой и проблему
- блокировка тривиального container escape
- снижение ущерба от RCE



UID/GID Mapping

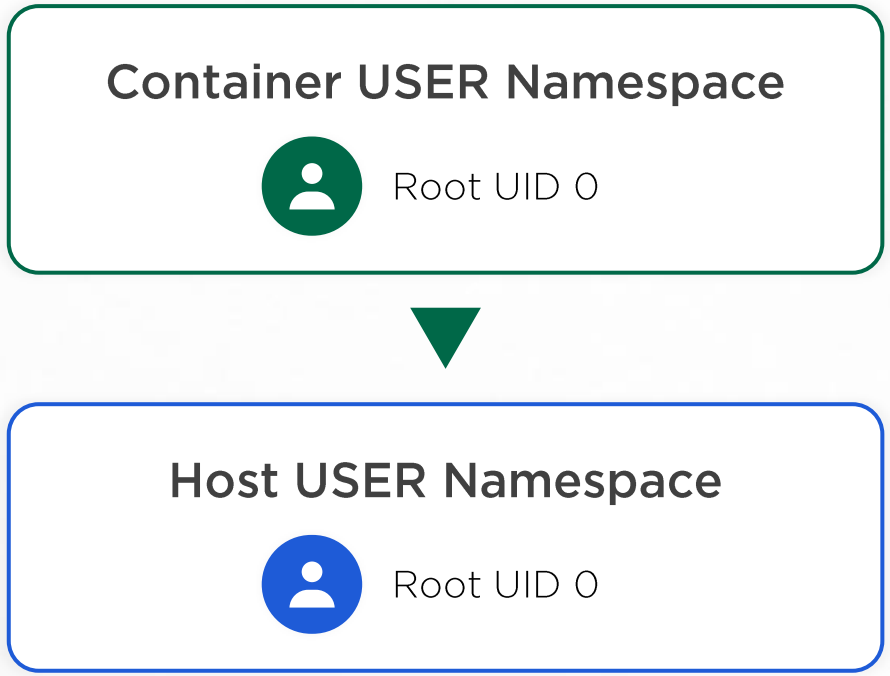
Ядро Linux по умолчанию переводит идентификаторы пользователей (UID) и групп (GID) из User Namespace контейнера в User Namespace хоста для согласования прав доступа к ресурсам хоста.

Если злоумышленник сбежит из контейнера (через уязвимость в Docker), он окажется на хосте с правами root

---

Если нет возможности запустить приложение в контейнере от имени NON-ROOT используй механизм **User Namespace Remapping**





Rootfs в режиме read-only

Описание

Корневая файловая система контейнера монтируется с флагом ro (read-only). Процесс внутри контейнера не может записывать в /, /usr, /etc, /bin и другие системные директории

Минимизация

Если злоумышленник не может писать в /usr/bin, /bin, /lib, он не сможет закрепиться в контейнере или подменить легитимные утилиты (например, ps, ls, curl) на вредоносные



Image: distribution-less

Distroless

Это тип образа Docker, который содержит только необходимые компоненты для запуска приложения. Значительно снижая поверхность атаки

Для сравнения:

~ 120 MB
alpine

VS

~ 20 MB
distroless



раздел

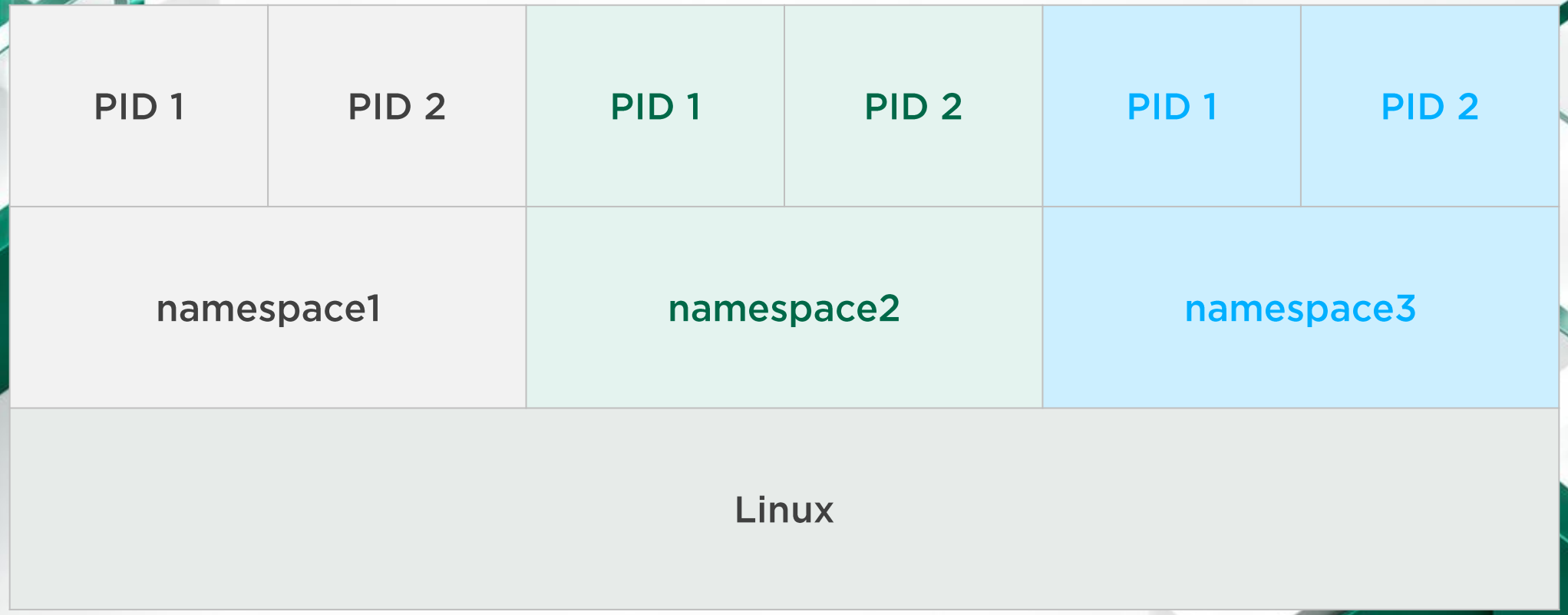
03

Namespaces и Cgroups



Namespaces

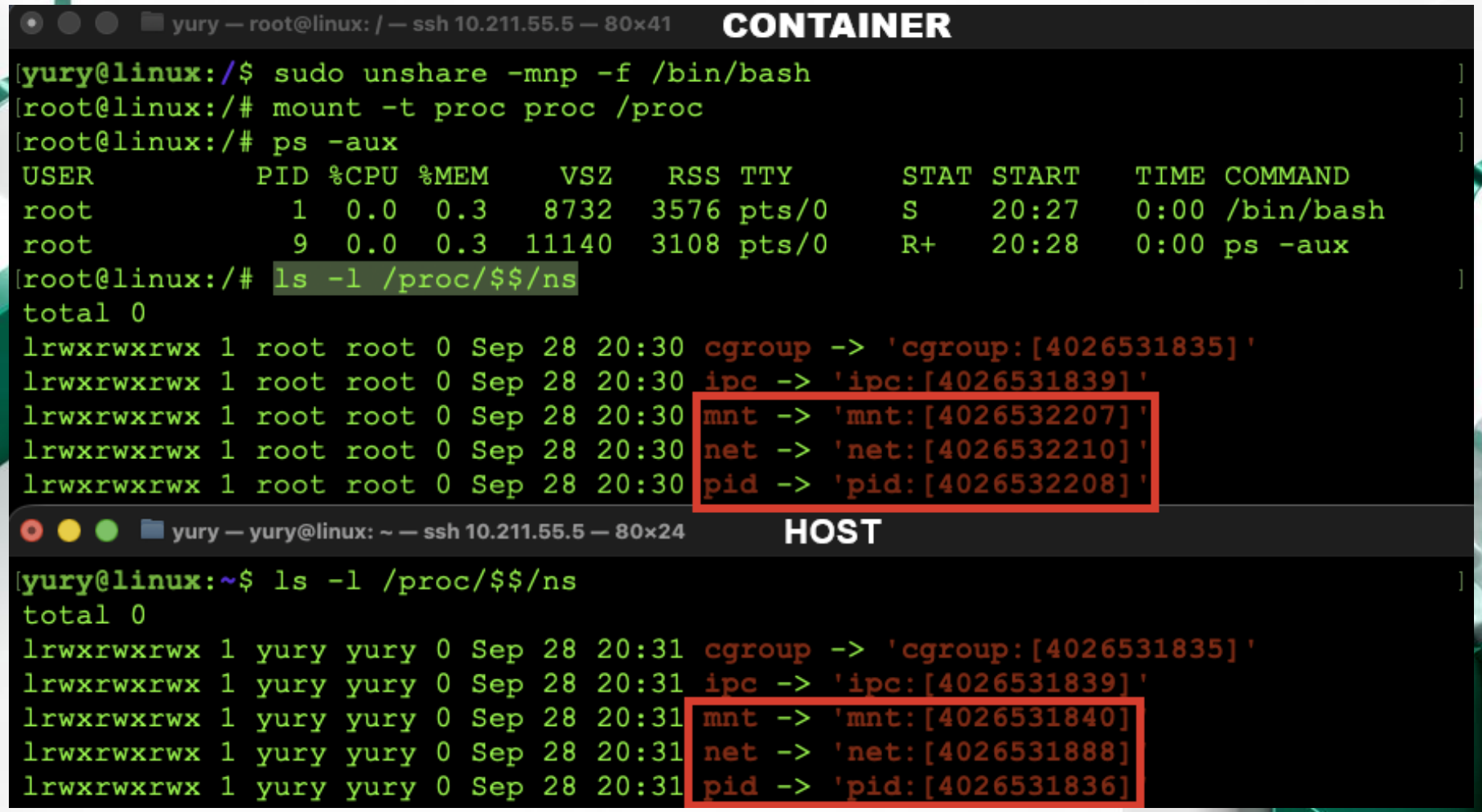
Namespaces — это механизм в ядре Linux, который обеспечивает изоляцию объектов для процессов



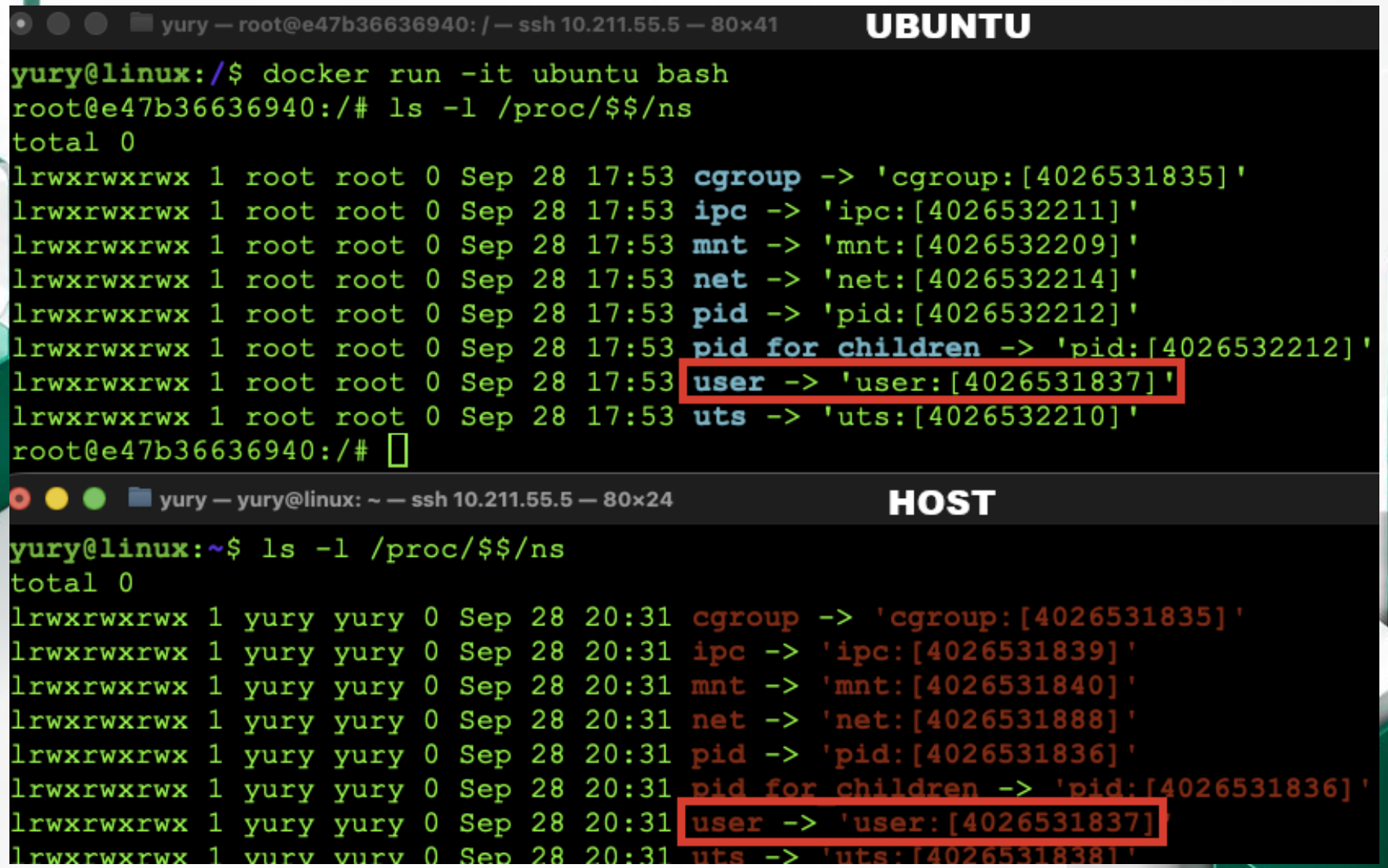
Namespaces

Namespace	Что изолирует	Эффект для процесса
01 PID	Список процессов	Процесс видит только свои процессы
02 Network	Сетевые интерфейсы, IP, порты	Процесс видит только свои сетевые интерфейсы
03 Mount	Точки монтирования файловой системы	Процесс видит свою файловую систему. Корень / — это корень контейнера, а не хоста
04 UTS	Имя хоста и домен	Процесс видит свой hostname (например, my-container), а не имя хоста. Для него это отдельный «хост» со своим именем
05 IPC	Межпроцессорное взаимодействие	Процесс видит только свои очереди сообщений
06 User	Пользователи и группы (UID/GID)	Процесс может быть root (UID 0) внутри контейнера, но на хосте это обычный user с UID 1000. Про
07 Cgroup	Видимость контрольных групп	Процесс видит только свою иерархию cgroups, а не всю систему. Он не знает, какие ещё cgroups есть на хосте и какие лимиты у соседей
08 Time	Системные часы	Процесс видит своё смещение монотонных и загрузочных часов. Это позволяет изолировать время внутри контейнера от хоста

Создание изолированного процесса



Запуск контейнера



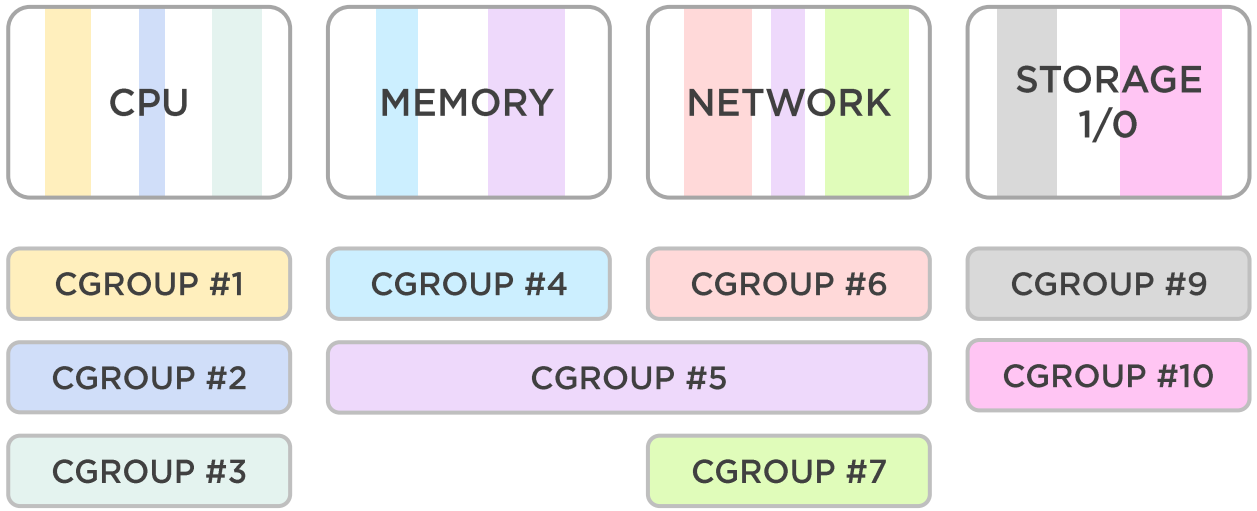
Cgroups

Cgroups (control groups) — это механизм ядра Linux, который позволяет ограничивать, учитывать и изолировать потребление ресурсов (CPU, память, диск, сеть, количество процессов) для группы процессов

Docker Grounds up: Resource Isolation

Cgroups: Isolation and accounting

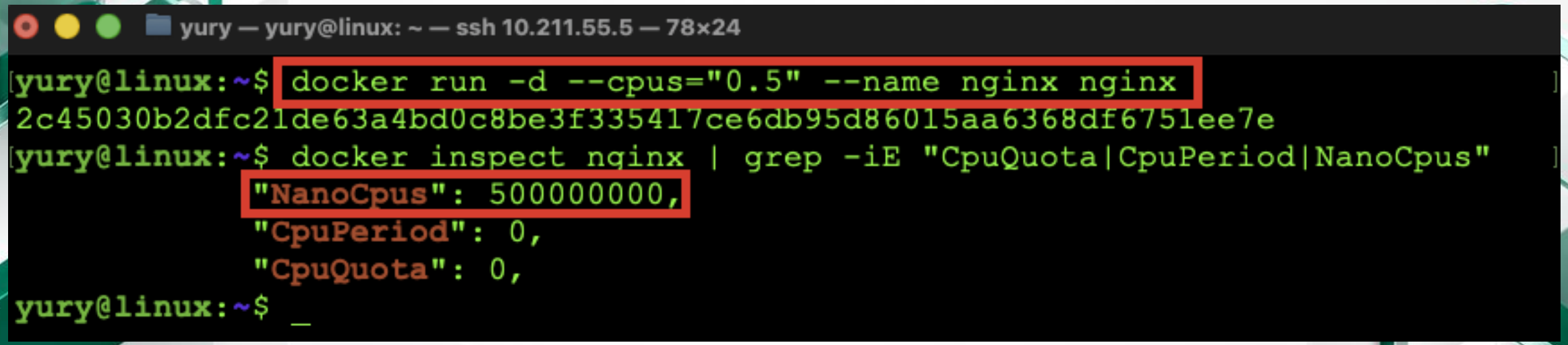
- cpu
- memory
- block i/o
- devices
- network
- numa
- freezer



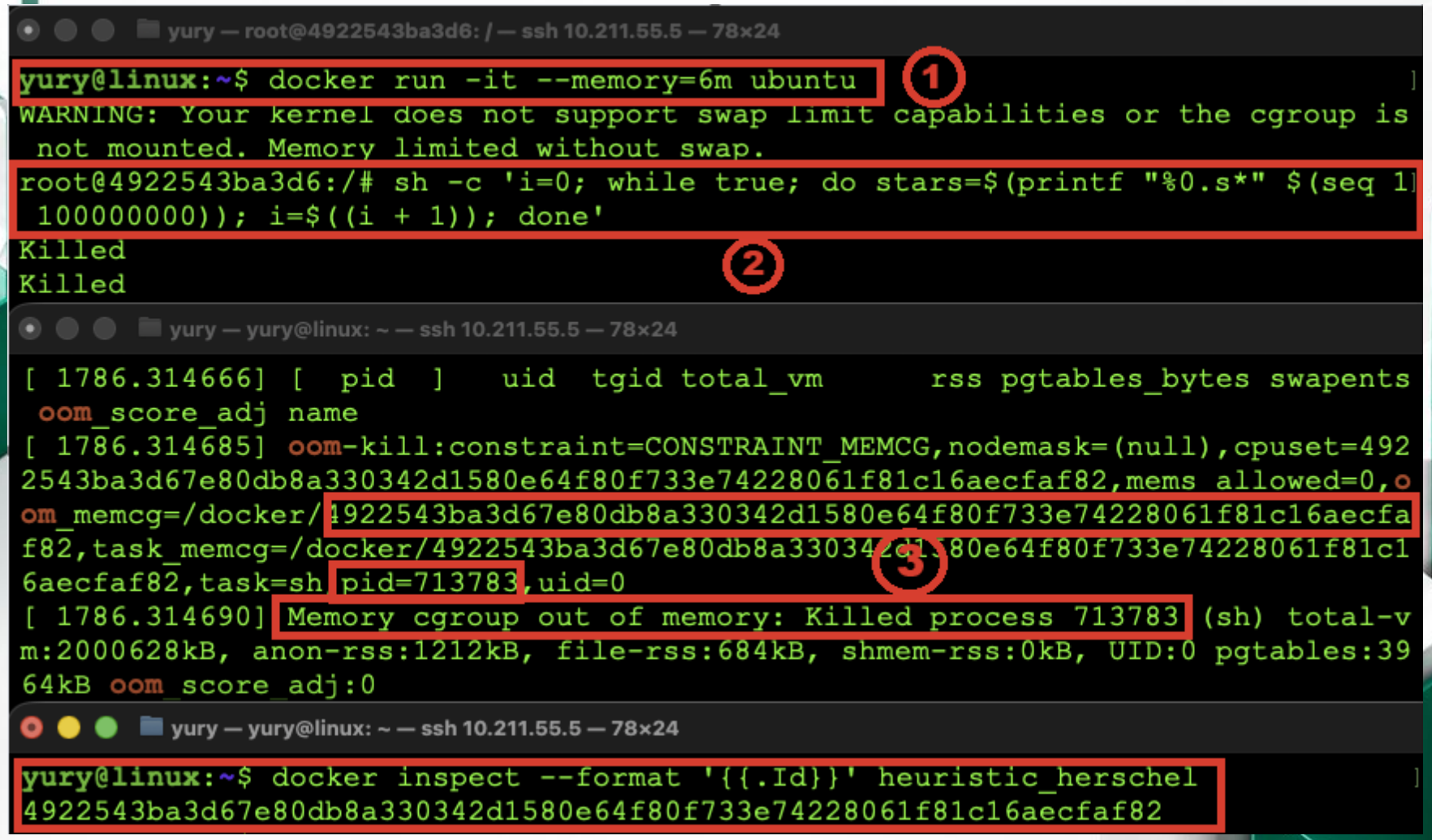
Cgroups

Cgroups	Что контролирует	Эффект
01 cpu	Процессорное время	Процесс получает не более заданной доли CPU
02 cpuset	Привязка к конкретным ядрам	Процесс видит и может выполняться только на разрешённых ядрах
03 memory	Оперативная память	Процесс не может получить больше заданного лимита. При превышении — OOM убивает процесс
04 blkio	Дисковый ввод-вывод	Процесс не может читать/писать быстрее заданного количества операций ввода-вывода в секунду
05 pids	Количество процессов/потоков	Защита от fork-бомб (вредоносная программа, которая бесконечно создаёт свои копии)
06 devices	Доступ к устройствам	Процесс не видит и не может использовать запрещённые устройства

Cgroup CPU


yury — yury@linux: ~ — ssh 10.211.55.5 — 78×24
yury@linux:~$ docker run -d --cpus="0.5" --name nginx nginx
2c45030b2dfc21de63a4bd0c8be3f335417ce6db95d86015aa6368df6751ee7e
yury@linux:~$ docker inspect nginx | grep -iE "CpuQuota|CpuPeriod|NanoCpus"
"NanoCpus": 500000000,
"CpuPeriod": 0,
"CpuQuota": 0,
yury@linux:~$ _

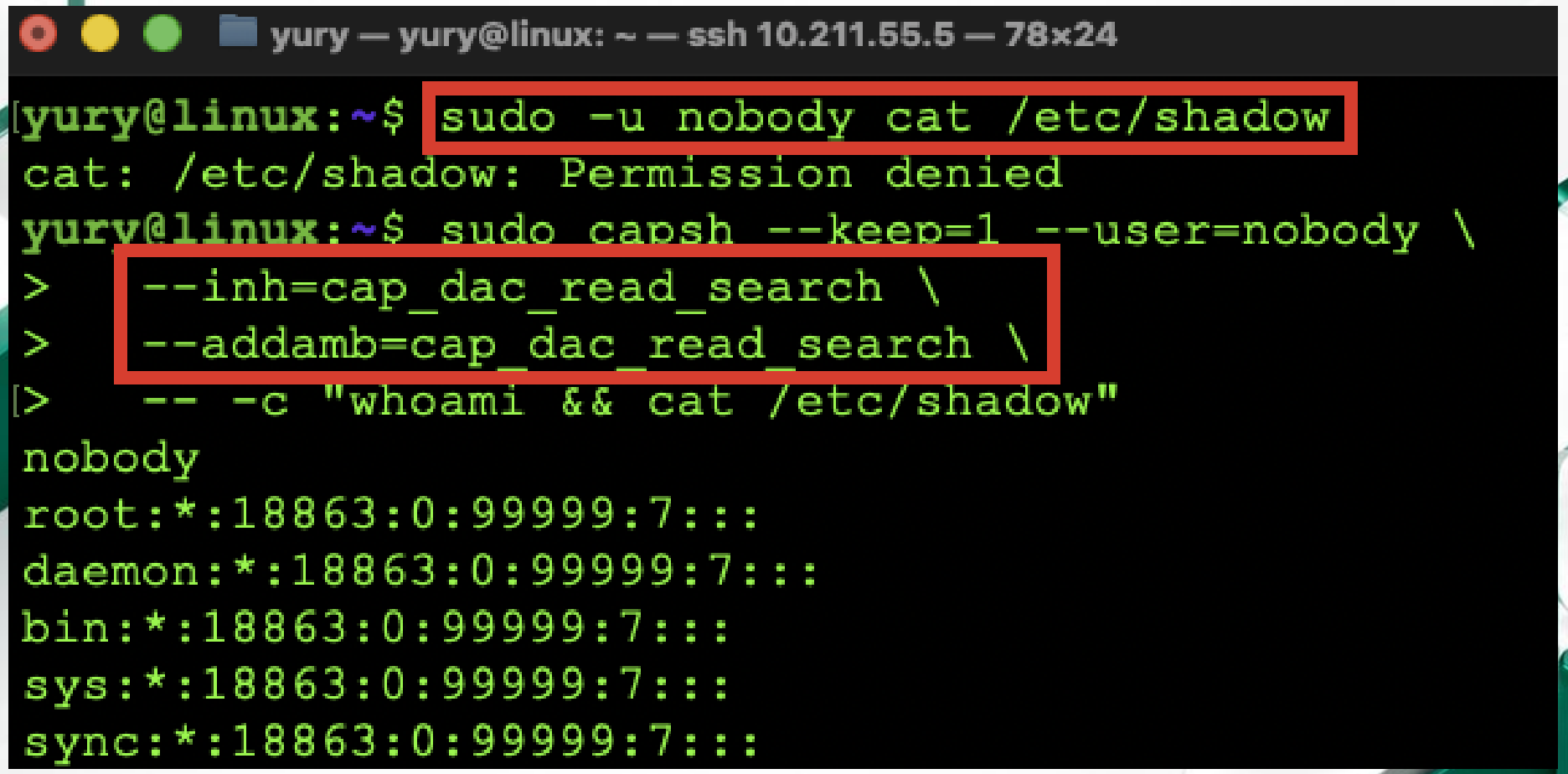
Cgroup memory: oom->pid=713783



Capabilities

Capabilities — это механизм ядра Linux, который позволяет передать процессу контейнера «только часть прав root'а», а не весь root целиком

Capabilities



Capabilities: наиболее опасные

Capability	Эффект	Риск
01 CAP_SYS_ADMIN	Процесс получает «новый root» — может монтировать файловые системы хоста, менять изоляцию, управлять другими контейнерами	Container escape, полный контроль над хостом
02 CAP_DAC_READ_SEARCH	Процесс читает любые файлы, включая /etc/shadow, ключи, секреты, даже если запущен от nobody	Кража секретов, чтение данных хоста
03 CAP_SYS_PTRACE	Процесс подключается к любому процессу, читает его память, внедряет код	Компрометация процессов хоста, кража токенов
04 CAP_SYS_MODULE	Процесс загружает вредоносный модуль в общее ядро	Container escape, выполнение кода в ядре
05 CAP_DAC_OVERRIDE	Процесс игнорирует права на чтение/запись любых файлов, даже от nobody	Изменение системных файлов, эскалация привилегий

Capabilities Docker

yury — root@769850f4b206: / — ssh 10.211.55.5 — 78×24

```
[root@769850f4b206:/# capsh --print                                    ]
Current: cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,
cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,cap_mkn
od,cap_audit_write,cap_setfcap=ep
Bounding set =cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_se
tgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,ca
p_mknod,cap_audit_write,cap_setfcap
Ambient set =
Current IAB: !cap_dac_read_search,!cap_linux_immutable,!cap_net_broadcast,!cap
_net_admin,!cap_ipc_lock,!cap_ipc_owner,!cap_sys_module,!cap_sys_rawio,!cap_sy
s_ptrace,!cap_sys_pacct,!cap_sys_admin,!cap_sys_boot,!cap_sys_nice,!cap_sys_re
source,!cap_sys_time,!cap_sys_tty_config,!cap_lease,!cap_audit_control,!cap_ma
c_override,!cap_mac_admin,!cap_syslog,!cap_wake_alarm,!cap_block_suspend,!cap_
audit_read
```

Best Practice: cap-drop=ALL

```
yury — root@a40a9fd0cacb: / — ssh 10.211.55.5 — 78×24
yury@linux:~$ docker run -it --cap-drop=ALL \
>    --cap-add=DAC_OVERRIDE \
>    --cap-add=CHOWN \
>    --cap-add=FOWNER \
>    --cap-add=SETGID \
>    --cap-add=SETUID \
>    ubuntu
root@a40a9fd0cacb:/# capsh --print
Current: cap_chown,cap_dac_override,cap_fowner,cap_setgid,cap_setuid=ep
Bounding set =cap_chown,cap_dac_override,cap_fowner,cap_setgid,cap_setuid
Ambient set =
Current IAB: !cap_dac_read_search,!cap_fsetid,!cap_kill,!cap_setpcap,!cap_linu
x_immutable,!cap_net_bind_service,!cap_net_broadcast,!cap_net_admin,!cap_net_r
aw,!cap_ipc_lock,!cap_ipc_owner,!cap_sys_module,!cap_sys_rawio,!cap_sys_chroot
,!cap_sys_ptrace,!cap_sys_pacct,!cap_sys_admin,!cap_sys_boot,!cap_sys_nice,!ca
p_sys_resource,!cap_sys_time,!cap_sys_tty_config,!cap_mknod,!cap_lease,!cap_au
dit_write,!cap_audit_control,!cap_setfcap,!cap_mac_override,!cap_mac_admin,!ca
p_syslog,!cap_wake_alarm,!cap_block_suspend,!cap_audit_read
```

раздел

04

Seccomp и AppArmor



НСПК | МИР | СБП

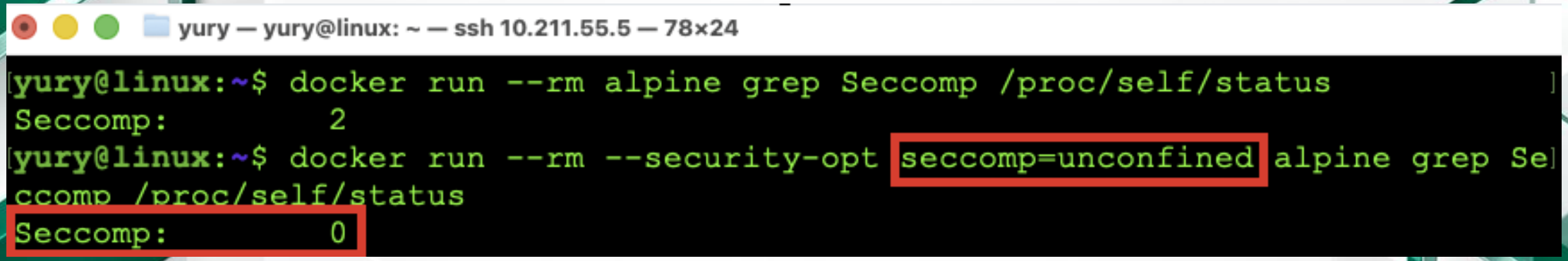
Seccomp

Seccomp (secure computing mode) — это механизм ядра Linux, который позволяет ограничить набор системных вызовов (syscalls), доступных процессу. Если процесс попытается выполнить запрещённый системный вызов, ядро его заблокирует

Syscalls (блокируются при запуске контейнера Docker)

Syscall	Риск	Описание
01 mount	Container Escape	Монтирование ФС хоста
02 ptrace	Privilege Escalation	Чтение памяти и внедрение кода в другие процессы (кража секретов)
03 init_module	Kernel Modules	Загрузка вредоносного модуля в общее ядро — полный контроль над хостом
04 keyctl	Kernel Keyring	Доступ к связке ключей ядра — кража секретов в обход namespaces
05 perf_event_open	Info Leak	Утечка адресов ядра и структур

Seccomp: старт контейнера с/без профиля



Seccomp: default docker profile

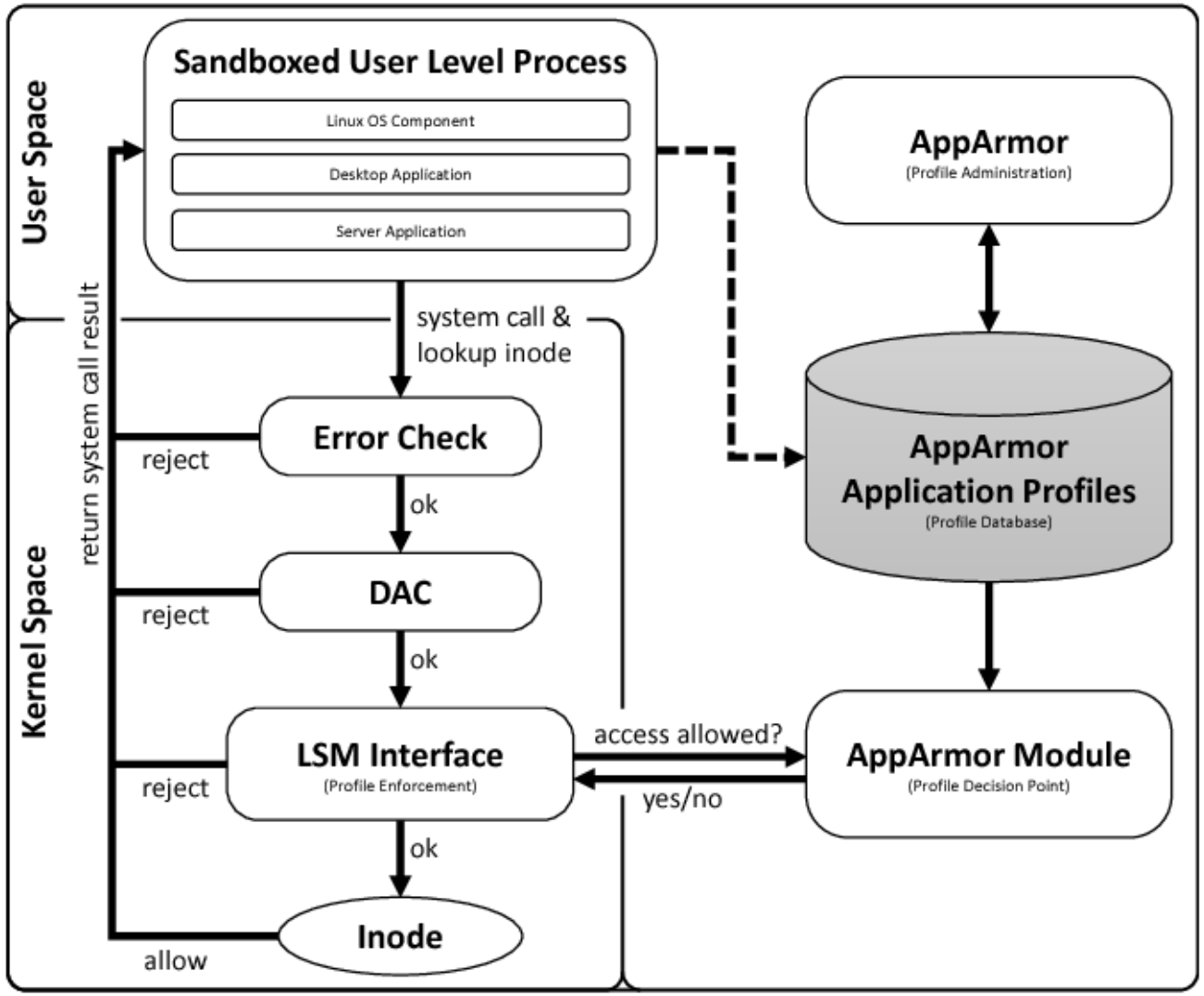
```
"defaultAction": "SCMP_ACT_ERRNO",
"defaultErrnoRet": 1,
"archMap": [
        {
                "architecture": "SCMP_ARCH_X86_64",
                "subArchitectures": [
                        "SCMP_ARCH_X86",
                        "SCMP_ARCH_X32"
                ]
```

```
"syscalls": [
        {
                "names": [
                        "accept",
                        "accept4",
                        "access",
                        "adjtimex",
                        "alarm",
```

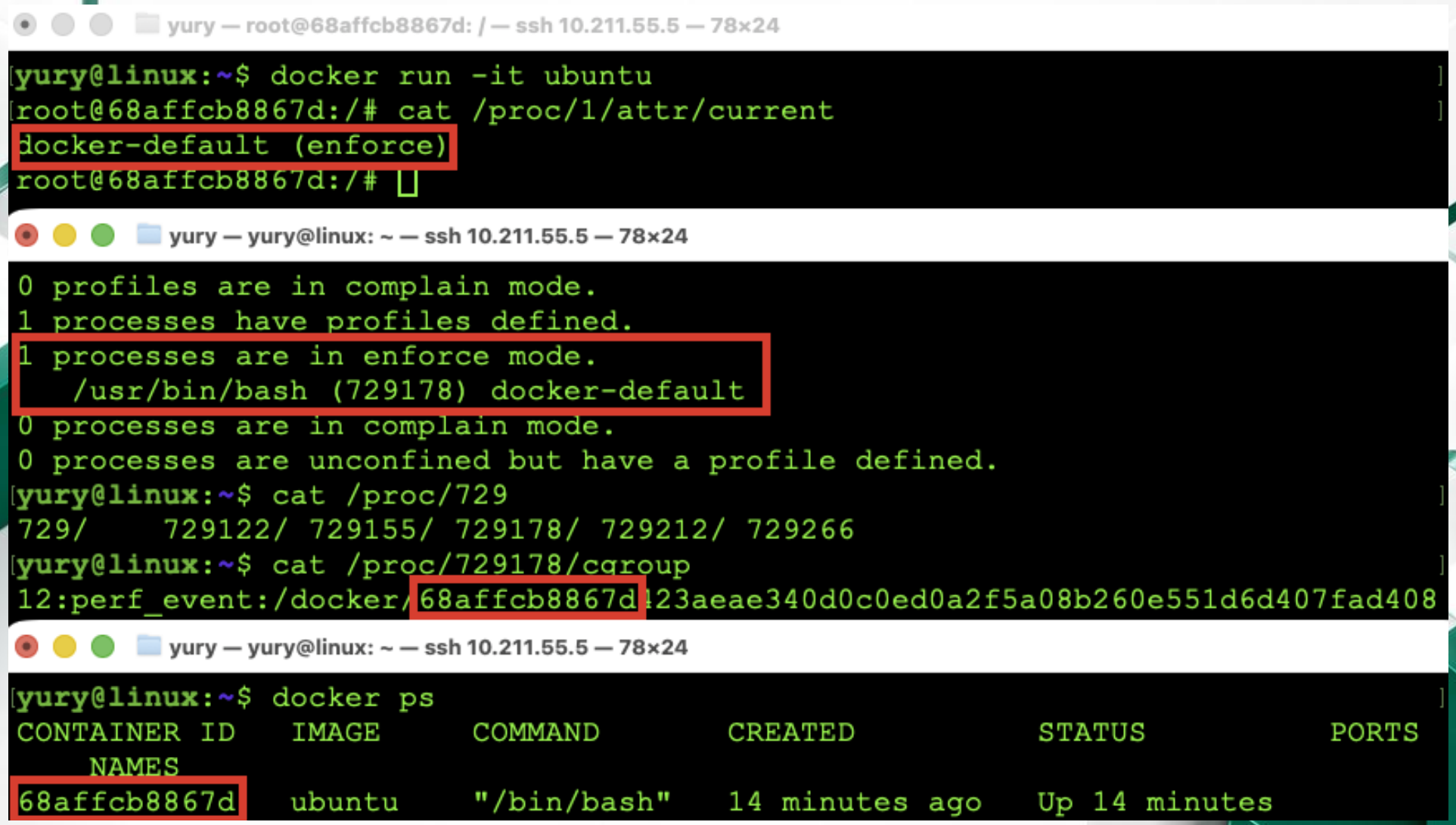
Apparmor

AppArmor (Application Armor) — это модуль безопасности ядра Linux (Linux Security Module, LSM), который реализует мандатное управление доступом (MAC)

Apparmor


User Space
Kernel Space
Sandboxed User Level Process
Linux OS Component
Desktop Application
Server Application
system call &
lookup inode
return system call result
Error Check
reject
ok
DAC
reject
ok
LSM Interface
(Profile Enforcement)
reject
ok
Inode
allow
AppArmor
(Profile Administration)
AppArmor
Application Profiles
(Profile Database)
access allowed?
yes/no
AppArmor Module
(Profile Decision Point)

AppArmor: default



Apparmor vs seccomp

Аспект	Seccomp	AppArmor
Объект контроля	Системные вызовы (syscalls) — какие функции ядра процесс может вызывать	Ресурсы: доступ к файлам, директориям, сети, capabilities
Принцип работы	Проверяет номер syscall и его аргументы до выполнения	Мандатное управление доступом (MAC). Проверяет путь к файлу или сетевой операции
Тип	Инструмент уменьшения поверхности атаки (не песочница)	Мандатная модель (MAC) — ужесточает права даже для root

Пример профиля AppArmor

```
#include <tunables/global>

profile secure-app /usr/bin/my-app flags=(attach_disconnected,
mediate_deleted) {
  #include <abstractions/base>

  # 1. CAPABILITIES: Разрешаем приложению менять UID/GID
  capability setuid,
  capability setgid,

  # 2. СЕТЬ: Разрешаем только TCP/IPv4 и UDP/IPv4
  network inet stream,
  network inet dgram,

  # 3. ФАЙЛЫ И ДИРЕКТОРИИ: Правила доступа
  # Чтение конфигурации
  /etc/my-app/** r,

  # Чтение и запись логов (только для файлов, принадлежащих
пользователю)
  owner /var/log/my-app/*.log rw,

  # Чтение и выполнение бинарников из директории приложения
  /opt/my-app/bin/** rix,
```

раздел 05

Инструменты безопасности. Итоги

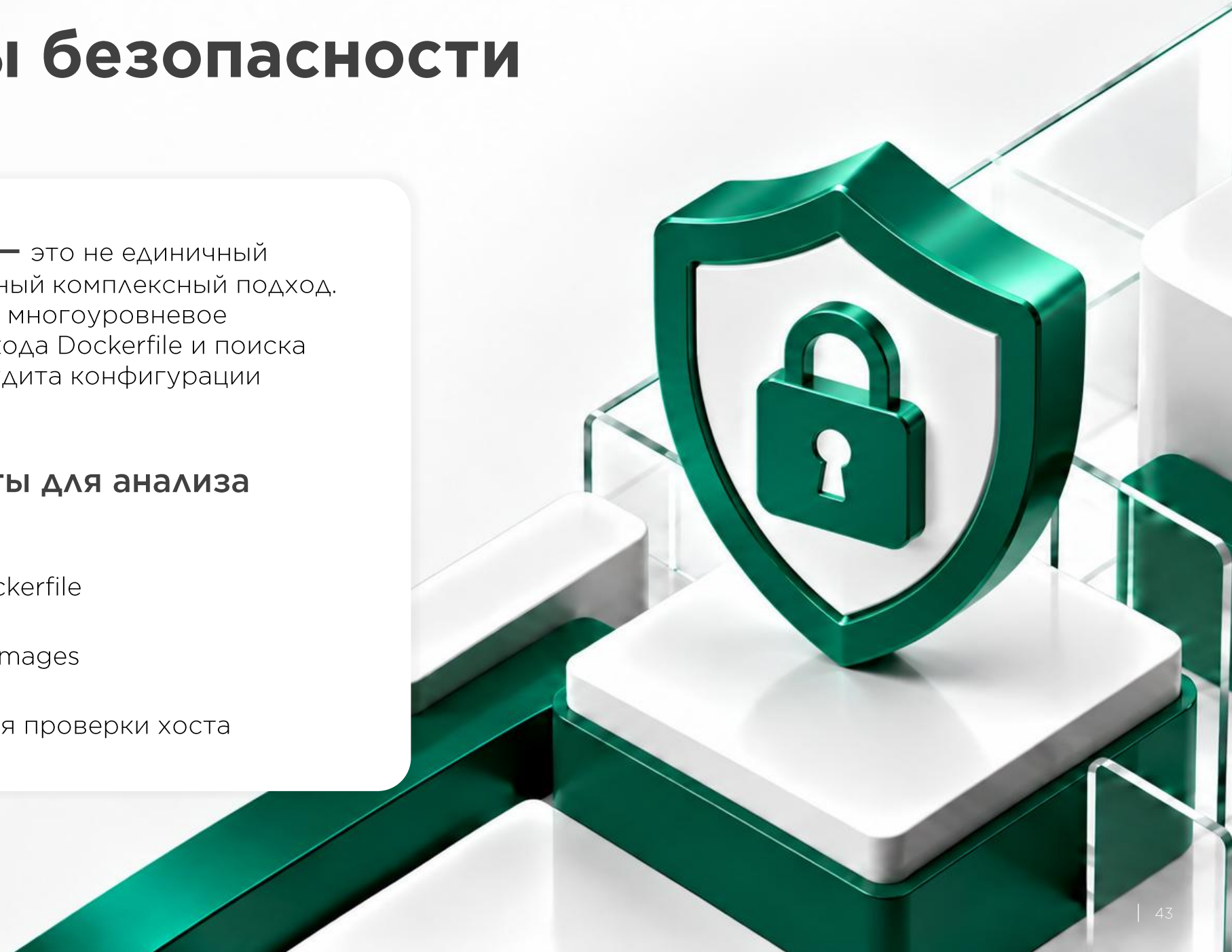


Инструменты безопасности

Безопасность Docker — это не единичный инструмент, а интегрированный комплексный подход. Ключевую роль в нём играет многоуровневое сканирование: от проверки кода Dockerfile и поиска уязвимостей в образах до аудита конфигурации runtime

Основные инструменты для анализа безопасности Docker:

- Hadolint для анализа Dockerfile
- Trivy для сканирования images
- Docker Bench Security для проверки хоста



Hadolint: линтер для docker

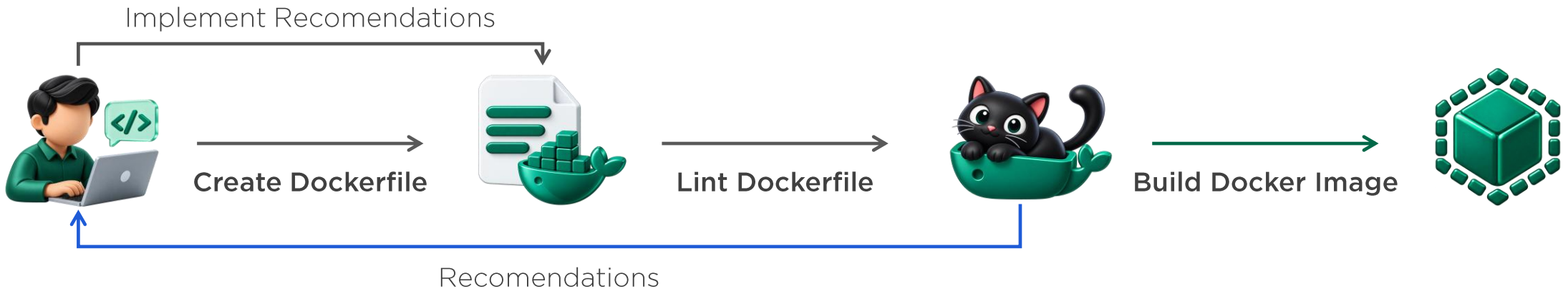
Hadolint — это статический анализатор для проверки Dockerfile на соответствие лучшим практикам Docker

Зачем он нужен?

Предотвращает создание и дальнейшее использование небезопасных образов

Выявляет:

- запуск контейнеров от имени root
- использование последних тегов (latest)
- удаления кэша менеджеров пакетов
- неоптимальное кеширование слоев



Hadolint in general

```
dockerdiver@linux:~$ docker run --rm -i hadolint/hadolint < Dockerfile
-:1 DL3007 warning: Using latest is prone to errors if the image will e
-:3 DL3015 info: Avoid additional packages by specifying `--no-install-
-:3 DL3009 info: Delete the apt lists (/var/lib/apt/lists) after instal
-:3 DL3008 warning: Pin versions in apt get install. Instead of `apt-ge
-:11 DL3002 warning: Last USER should not be root
```

- **DL3007** — использование latest может привести к ошибкам при обновлении образа
- **DL3015** — избегайте установки дополнительных пакетов, указав параметр --no-install-recommends
- **DL3009** — удаляйте списки apt (/var/lib/apt/lists) после установки списков пакетов для уменьшения размера образа
- **DL3008** — фиксируйте версии в apt-get install
- **DL3002** — не запускайте контейнер от пользователя root

```
dockerdiver@linux:~$ cat Dockerfile
FROM ubuntu:latest

RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    curl \
    wget \
    vim \
    git

USER root

COPY . /app

RUN pip3 install --no-cache-dir -r /app/requ

EXPOSE 80

CMD ["python3", "/app/app.py"]
```

Trivy: сканер для уязвимостей

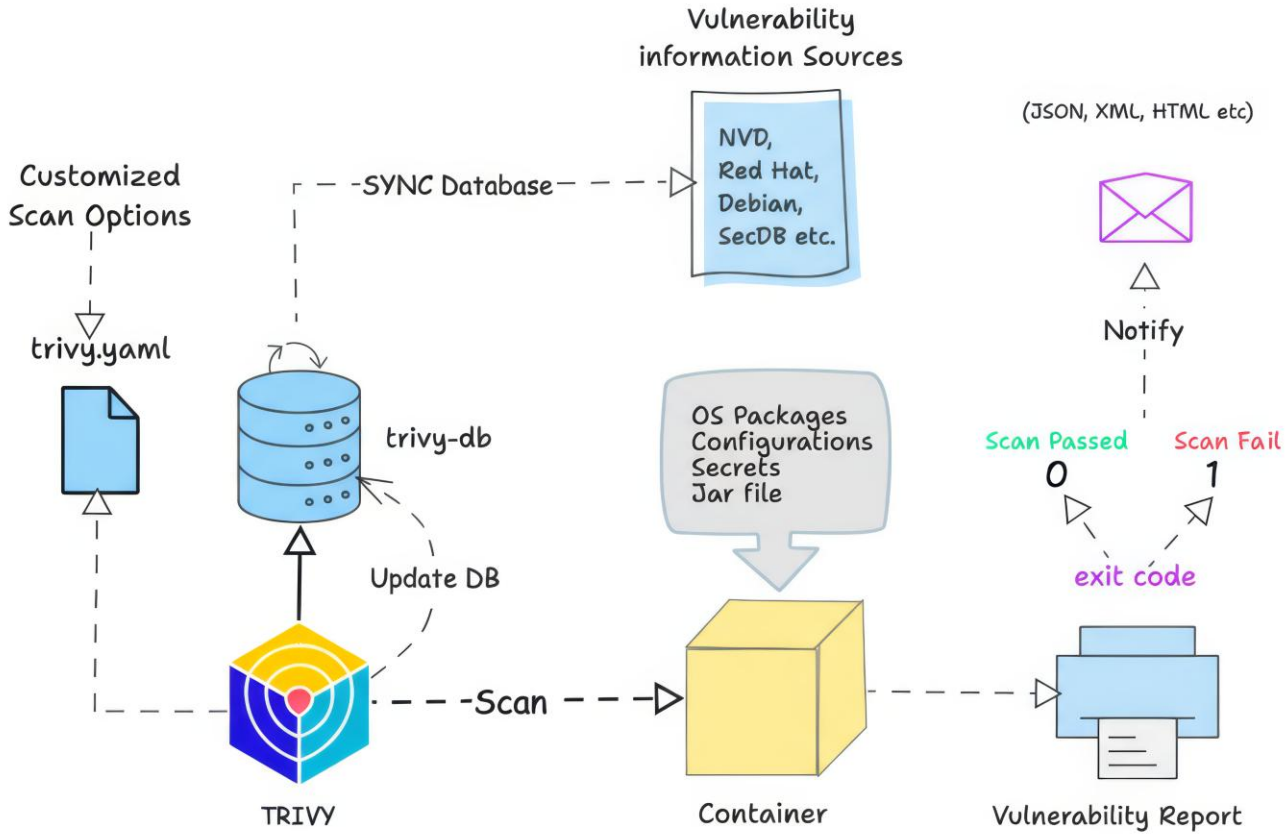
Trivy — сканер уязвимостей с открытым исходным кодом. Требует отдельной загрузки базы данных уязвимостей

Зачем он нужен?

Обеспечивает "сканирование как код" (Scanning as Code), легко интегрируется в любой CI-пайплайн для автоматической проверки собранного образа

Применяется для сканирования:

- образов контейнеров на наличие известных уязвимостей (CVE) в установленных пакетах
- файловых системы и Git-репозитории
- конфигурации (Dockerfile) на соответствие лучшим практикам CIS Benchmark



Trivy vs docker scout in general

```
root@linux:~# trivy image nginxinc/nginx-unprivileged:latest
2025-10-17T20:49:51+03:00       INFO    [vuln] Vulnerability scanning is enabled
2025-10-17T20:49:51+03:00       INFO    [secret] Secret scanning is enabled
2025-10-17T20:49:51+03:00       INFO    [secret] If your scanning is slow, please try '--
2025-10-17T20:49:51+03:00       INFO    [secret] Please see https://trivy.dev/v0.67/docs/
Report Summary
```

Target	Type	Vulnerabilities	Secrets
nginxinc/nginx-unprivileged:latest (debian 13.1)	debian	93	-

```
nginxinc/nginx-unprivileged:latest (debian 13.1)

Total: 93 (UNKNOWN: 0, LOW: 82, MEDIUM: 10, HIGH: 1, CRITICAL: 0)
```

Сравним выводы для сканирования образа trivy vs docker scout:

```
                                  Analyzed Image
Target                  nginxinc/nginx-unprivileged:latest
  digest                455b6483a814
  platform              linux/arm64
  provenance            https://github.com/nginx/docker-nginx-unprivileged.git#c
                        cf4e6821f3dec2cb24bc3156251810beb3ea4187
  vulnerabilities       0C  1H  1M  44L
  size                  62 MB
  packages              232

Base image              debian:trixie-slim
                        1caf1c703c8f
```

!

По результатам сравнения trivy показал больше уязвимостей. Сканеры определили уровень критичности «High» для пакета libxslt

Docker Bench for Security

DBS — это официальный скрипт на языке Bash для автоматизации проверки хоста с Docker на соответствие **CIS Docker Benchmark 1.6**

Применяется для сканирования:

- конфигурация хоста
- конфигурация Docker-демона
- конфигурации контейнеров Docker
- безопасность Docker-образов
- безопасность Runtime контейнеров

```
dockerdiver@linux:~/docker-bench-security$ docker run --rm --net host --pid h
> -e DOCKER_CONTENT_TRUST=$DOCKER_CONTENT_TRUST \
> -v /etc:/etc:ro \
> -v /usr/bin/containerd:/usr/bin/containerd:ro \
```

```
[WARN]       * Container running without CPU restrictions: stoic_pasteur
[WARN] 5.13 - Ensure that the container's root filesystem is mounted as read
[WARN]       * Container running with root FS mounted R/W: stoic_pasteur
[WARN] 5.14 - Ensure that incoming container traffic is bound to a specific h
[WARN]       * Port being bound to wildcard IP: 0.0.0.0 in stoic_pasteur
0
[PASS] 5.15 - Ensure that the 'on-failure' container restart policy is set to
[PASS] 5.16 - Ensure that the host's process namespace is not shared (Automate
[PASS] 5.17 - Ensure that the host's IPC namespace is not shared (Automated)
[PASS] 5.18 - Ensure that host devices are not directly exposed to containers
```

Итоги

docker run -d --name my-secure-app

```
 --user 1000:1000                                          # Запуск от непривилегированного пользователя
  --read-only                                              # Read-only корневая ФС
  --tmpfs /var/run                                         # Writeable /var/run
--cap-drop=ALL                                             # Удаляем все capabilities
  --cap-add=NET_BIND_SERVICE                               # Добавляем только нужную capabilities
--security-opt no-new-privileges:true                      # Запрещаем повышение привилегий
--security-opt apparmor=docker-default                     # Подключаем профиль AppArmor
--security-opt seccomp=/path/to/seccomp/profile.json       # Подключаем профиль Seccomp
--pids-limit 100                                           # Ограничиваем число процессов
-m 512m                                                    # Ограничиваем память
--cpus 1.5                                                 # Ограничиваем CPU
-v /host/path/config:/app/config:ro                        # Монтируем volume только для чтения
  -p 8080:80                                               # Публикуем только нужный порт
my-scanned-app:latest
```

Критичные объекты:

- /var/lib/docker
- /etc/docker
- Docker.service
- Docker.socket
- /etc/default/docker
- /etc/docker/daemon.json
- /etc/sysconfig/docker
- /usr/bin/containerd
- /usr/sbin/runc

ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

Спасибо
за внимание!

Юрий
Паркин



НСПК | МИР | СБП